

The Online Market.

What is the average price breakdown of a laptop

1.1 Background

A laptop is a portable computer designed for easy transport. It is intended for personal use and can perform a wide range of functions, including internet browsing, document creation, and gaming. Laptops are typically smaller than desktop computers and operate on battery power.

1.2 History of laptops

The development of the laptop parallels the evolution of the personal computer. Alan Kay envisioned a portable device in 1968, described as the "Dynabook" in 1972. IBM's SCAMP prototype in 1973 led to the IBM 5100, the first portable computer, in 1975. The rise of 8-bit CPUs saw an increase in portable computers. The Epson HX-20, the first "laptop-sized notebook," appeared in 1982. The GRiD Compass 1101, introduced in 1982, was used by NASA and the military.

Innovations like touchpads (1983), pointing sticks (1992), and handwriting recognition (1987) emerged. The 1980s saw laptops with red plasma displays needing AC power. The 1990s brought power-efficient CPUs and memory cards. VGA resolution became standard by 1988, with color screens common by 1991. Hard drives became prevalent by 1990. Webcams reached 720p in early laptops, with 1080p models appearing in the 2010s. Optical drives were popular from 1997 but declined by 2011, though they remain available as external peripherals.

1.3 Rise of e-commerce

In recent years, e-commerce has become an integral part of global retail. The buying and selling of goods have significantly transformed with the advent of the internet. The ongoing digitalization of modern life allows consumers worldwide to benefit from online transactions. As global internet access and adoption rapidly increase, with over five billion internet users, the number of people making online purchases continues to grow. In 2024, retail e-commerce sales are projected to exceed 6.3 trillion U.S. dollars globally, with expectations for continued growth in the coming years.

1.4 Why Laptops?

Analyzing the competitive market for laptop sales has been an ongoing effort, given the market's constant evolution. As an avid laptop user, I am inspired to research this topic. Leveraging the knowledge gained from this course, I am examining data from

the internet to determine which laptop components can be optimized for cost savings, where to find the most affordable prices, and which laptop models are currently popular.

1.5 Aims and Objectives

1.5.1 Aims:

- Analyze the competitive laptop market - What defines a successful laptop? What is the process to achieve cost-effective production?
- Identify factors that make a laptop attractive to consumers
- Create a checklist to guide potential buyers in choosing popular and cost-efficient laptops

1.5.2 Objectives:

- Research current laptop ratings and reviews
- Analyze the competitive landscape of laptop sales
- Filter through online datasets to identify trends that make certain laptops more popular
- Examine the relationship between consumer feedback and the popularity of laptops

1.6 Data and Limitations

1.6.1 Data

The eBay and Amazon datasets were adapted from Kaggle users KEREM ABDULLAHOĞLU and TALHA BARKAAT AHMAD, respectively. Both datasets are relatively up-to-date, with eBay data from April 2024 and Amazon data from October 2023.

KEREM ABDULLAHOĞLU obtained the eBay dataset using a web API. It includes not only new, factory-sealed products but also refurbished or second-hand items. This dataset consists of 3,981 rows and 10 distinct columns.

TALHA BARKAAT AHMAD scraped the Amazon.com dataset. It contains product details such as rating, price, operating system, title, review count, and display size. This dataset comprises approximately 4.5K data points, each representing a unique laptop listing, and includes 14 distinct features.

I primarily used and cleaned the dataset from eBay; however, I dropped some columns from the Amazon dataset and adapted it to align with the eBay dataset for a direct comparison.

1.6.2 Limitations

Despite the presence of multiple online stores, the laptop market in e-commerce is largely monopolized by a few major sites. This dominance is mainly due to the high cost of laptops, leading consumers to prefer more reputable sites for their purchases.

1.7 Ethics of Data:

Based on Kaggle's terms and conditions, I am permitted to use the datasets as long as the authors are credited. These datasets are utilized solely for this midterm project, and the inferences drawn from the data analysis are based on my personal knowledge and research, appropriately referenced.

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
```

2 eBay

2.1 Data Cleaning and Processing

I am loading and exploring the Ebay csv file using the pandas library. i first examine the shape and the summary on the dataframe to understand what has to be done to clean it at a glance

```
In [ ]: ebay_df = pd.read_csv('Ebay.csv')
ebay_df.shape
```

```
Out[ ]: (3981, 10)
```

```
In [ ]: ebay_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3981 entries, 0 to 3980
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Brand                  3949 non-null   object
1   Product_Description    3980 non-null   object
2   Screen_Size            3785 non-null   object
3   RAM                    3730 non-null   object
4   Processor              3880 non-null   object
5   GPU                    3247 non-null   object
6   GPU_Type              3258 non-null   object
7   Resolution             3320 non-null   object
8   Condition              3981 non-null   object
9   Price                  3980 non-null   float64
dtypes: float64(1), object(9)
memory usage: 311.1+ KB
```

2.1.1 Screen Size

the code below checks if the rows under the column screen sizes are missing and fills it in by checking a pattern in the csv file and matches it

```
In [ ]: import re
def filterScreenSize(title):
    pattern = r"(\d+(?:\.\d+)?)\s*['\""]"
    match = re.search(pattern, str(title))
    return match.group() if match else np.nan

ebay_df.loc[ebay_df['Screen_Size'].isna(), 'Screen_Size'] = ebay_df[ebay_
```

```
In [ ]: def changeNumericValue(title):
    pattern = r'[-+]?[0-9]*\.\?[0-9]+'
    match = re.search(pattern, str(title))
    if match:
        return match.group()
    else:
        return str(title)
```

```
In [ ]: ebay_df['Screen_Size'] = ebay_df['Screen_Size'].apply(changeNumericValue)
```

dropping the rows that i do not need or is an invalid number

```
In [ ]: mask = ebay_df['Screen_Size'] == 'nan'
ebay_df = ebay_df.drop(ebay_df[mask].index)
mask = ebay_df['Screen_Size'] == 'Does'
ebay_df = ebay_df.drop(ebay_df[mask].index)
mask = ebay_df['Screen_Size'] == 'Not'
ebay_df = ebay_df.drop(ebay_df[mask].index)
mask = ebay_df['Screen_Size'] == 'N\A'
ebay_df = ebay_df.drop(ebay_df[mask].index)
mask = ebay_df['Screen_Size'] == 'Unknown'
ebay_df = ebay_df.drop(ebay_df[mask].index)
mask = ebay_df['Screen_Size'] == '0'
ebay_df = ebay_df.drop(ebay_df[mask].index)
mask = ebay_df['Screen_Size'] == '39.6'
ebay_df = ebay_df.drop(ebay_df[mask].index)
mask = ebay_df['Screen_Size'] == '23.8'
ebay_df = ebay_df.drop(ebay_df[mask].index)

ebay_df.Screen_Size = pd.to_numeric(ebay_df.Screen_Size)
```

2.1.2 Ram Size

```
In [ ]: def filterRam(title):
    pattern = r"(\d+(?:\.\d+)?)\s*GB\s*RAM"
    match = re.search(pattern, title)
    return match.group(1) if match else np.nan

ebay_df.loc[ebay_df['RAM'].isna(), 'RAM'] = ebay_df[ebay_df['RAM'].isna()]
```

```
In [ ]: ebay_df['RAM'] = ebay_df['RAM'].str.lower().str.replace("gb", "")
ebay_df['RAM'] = ebay_df['RAM'].str.lower().str.replace("up", "")
```

```
ebay_df['RAM'] = ebay_df['RAM'].str.lower().str.replace("upto", "")
ebay_df['RAM'] = ebay_df['RAM'].str.lower().str.replace("to", "")
ebay_df['RAM'].unique()
```

```
Out[ ]: array(['16', '64', '', '8', nan, '32', '24', '4', '40', '128', '12',
              '4,',
              '512', '8,', '20', '256', '16,'], dtype=object)
```

```
In [ ]: ebay_df[ebay_df['RAM'].isna()].shape
```

```
Out[ ]: (135, 10)
```

Dropping N/A values and change to float

```
In [ ]: ebay_df.dropna(subset=['RAM'], inplace=True)
```

2.1.3 Brand

```
In [ ]: ebay_df[ebay_df['Brand'].isna()]
```

Out[]:

	Brand	Product_Description	Screen_Size	RAM	Processor	GPU	GP
120	NaN	Dell Precision 5520 15.6" Intel i5 8GB RAM 240...	15.6	8	Intel Core i5 7th Gen.	Intel HD Graphics 630	Integra C
241	NaN	Lenovo ThinkPad T470s 14" Intel i5 6th Gen 8GB...	14.0	8	Intel Core i5 6th Gen.	Intel HD Graphics 520	Integra C
317	NaN	Lenovo ThinkPad T480s 14" Intel i5 8th Gen 8GB...	14.0	8	Intel Core i5 8th Gen.	Intel UHD Graphics 620	Integra C
355	NaN	Lenovo IdeaPad 5 15ITL05 15.6" (512GB SSD, Int...	15.6	8	NaN	NaN	
427	NaN	Dell Inspiron 15 I3511- 5641BLK-PUS 15.6- Inch T...	15.6	16	Intel Core i5	NaN	
661	NaN	Dell Latitude 7490 14" PC Gaming Laptop - 16GB...	14.0	16	NaN	NaN	
794	NaN	Dell 14" Touchscreen Laptop Intel Core i7 1.9 ...	14.0	16	Intel Core i7	NaN	
825	NaN	Dell Latitude 3000 3330 13.3" Notebook DWXTG-O...	13.3	8	Intel Core i5	NaN	
859	NaN	Dell Latitude 3500 15.6" Intel i5 16GB RAM 500...	15.6	16	Intel Core i5 8th Gen.	Intel UHD Graphics 620	Integra C
892	NaN	Dell Latitude 3300 13" Intel i5 8GB RAM 120GB ...	13.3	8	Intel Core i5 8th Gen.	Intel UHD Graphics 620	Integra C
1073	NaN	Dell Latitude 3390 2- in-1 13" Intel i5 8GB RAM...	13.3	8	Intel Core i5 8th Gen.	Intel UHD Graphics 620	Integra C
1084	NaN	Dell Latitude 7480 14" Intel i7 6th Gen 16 GB ...	14.0	16	Intel Core i7 6th Gen.	Intel HD Graphics 520	Integra C
1480	NaN	Dell Latitude 5510 15" Intel i7 16GB RAM 500GB...	15.6	16	Intel Core i7 10th Gen.	NaN	Integra C
1546	NaN	Dell Latitude 7420 14" Intel i5 16GB RAM 240GB...	14.0	16	Intel Core i5 11th Gen.	Intel Iris Xe Graphics	Integra C
1686	NaN	Lenovo IdeaPad 3 15IAU7 82RK001KUS 14" Class N...	15.6	8	Intel Core i5	NaN	

	Brand	Product_Description	Screen_Size	RAM	Processor	GPU	GP
1844	NaN	Dell Latitude 7000 7420 14" Notebook KC1XR	14.0	16	Intel Core i7	NaN	
1998	NaN	Dell V5590- 7340GRY-PUS Vostro 5590 15.6-inch L...	15.6	16	Intel Core i7	NaN	
2017	NaN	Dell Latitude 7290 12.5" Intel i7 16GB RAM 500...	12.5	16	Intel Core i7 8th Gen.	NaN	Integra (C
2026	NaN	Lenovo IdeaPad 15.6" Laptop Intel Core i5 1.3 ...	15.6	8	Intel Core i5	NaN	
2040	NaN	Dell Inspiron 15 I3501-5450BLK-PUS 15.6-Inch L...	15.6	16	Intel Core i5	NaN	
2054	NaN	Dell Latitude 7000 7410 14" Notebook 9MP78	14.0	16	Intel Core i5	NaN	
2620	NaN	Dell Latitude 14" Laptop Intel Core i5 1.7 Ghz...	14.0	16	Intel Core i5	NaN	
2622	NaN	Dell Latitude 7400 Laptop 9VGPN	14.0	16	Intel Core i7	NaN	
2954	NaN	Lenovo IdeaPad 3i 82RK00SXUS 15.6- Inch Laptop	15.6	8	Intel Core i3	NaN	
3290	NaN	Dell Latitude 7000 7330 13.3" Notebook MYR12	13.3	16	Intel Core i5	NaN	
3699	NaN	Dell Latitude 7000 7320 13.3" Notebook 9PPWV	13.3	16	Intel Core i5	NaN	
3708	NaN	Dell Latitude 5000 5330 13.3" 2 in 1 Notebook ...	13.3	16	Intel Core i7	NaN	

Ignore the casing of the brands is the most important part of this component because if it is not in the proper case the compiler would not know how to identify the patterns and would not know how to search and replace the matches

```
In [ ]: def filterBrand(title):
        pattern = re.compile(r'(\lenovo|dell)', re.IGNORECASE)
        match = re.search(pattern, title)
        return match.group(1) if match else np.nan

ebay_df.loc[ebay_df['Brand'].isna(), 'Brand'] = ebay_df[ebay_df['Brand'].i
```

Some Brands have to be manually renamed

```
In [ ]: ebay_df['Brand'] = ebay_df['Brand'].replace('MICROSOFT', 'Microsoft')
ebay_df['Brand'] = ebay_df['Brand'].replace('Microsoft Surface', 'Microso
ebay_df['Brand'] = ebay_df['Brand'].replace('Dell gaming games game', 'De
ebay_df['Brand'] = ebay_df['Brand'].replace('SAMSUNG', 'Samsung')
ebay_df['Brand'] = ebay_df['Brand'].replace('LG Electronics', 'LG')
ebay_df['Brand'] = ebay_df['Brand'].replace('Hewlett Packard', 'HP')
ebay_df['Brand'] = ebay_df['Brand'].replace('VAIO', 'Sony')
ebay_df['Brand'] = ebay_df['Brand'].replace('Alienware', 'Dell')
ebay_df['Brand'] = ebay_df['Brand'].replace('Dell Latitude', 'Dell')
ebay_df['Brand'] = ebay_df['Brand'].replace('ThinkPad', 'Lenovo')
ebay_df['Brand'] = ebay_df['Brand'].replace('DELL', 'Dell')
ebay_df['Brand'] = ebay_df['Brand'].replace('Dell Inc', 'Dell')
ebay_df['Brand'] = ebay_df['Brand'].replace('LENOVO', 'Lenovo')
ebay_df['Brand'] = ebay_df['Brand'].replace('ASUS', 'Asus')
ebay_df['Brand'] = ebay_df['Brand'].replace('GIGABYTE', 'Gigabyte')
ebay_df['Brand'] = ebay_df['Brand'].replace('AORUS', 'Gigabyte')
```

2.1.4 Processors

I'm currently filtering and replacing values, but it's a bit complex. I'm focusing exclusively on Intel and AMD processors because only these two have a consistent system for displaying their specifications. I achieve this by searching for keywords like "Intel Core" and "Ryzen" or "R", and then systematically checking the subsequent numbers.

```
In [ ]: def parse_processor_info(processor):
# intel_pattern = r".*((Intel Core )?i[3579])-(?=[0-9]+).*"
intel_pattern = r".*((Intel Core )?i[3579])-((?=\d{5})\d{2}|(?=\d{4})"
amd_pattern = r".*(Ryzen|R)(\s*\d{1})\s*(\d{4})([A-Za-z])?.*"

intel_match = re.match(intel_pattern, str(processor), re.IGNORECASE)
amd_match = re.match(amd_pattern, str(processor), re.IGNORECASE)
if intel_match:
    if int(intel_match.group(3)[:2]) == 1:
        return f"{'Intel Core ' if intel_match.group(2) is None else
    if int(intel_match.group(3)[:2]) == 2:
        return f"{'Intel Core ' if intel_match.group(2) is None else

    if int(intel_match.group(3)[:2]) == 3:
        return f"{'Intel Core ' if intel_match.group(2) is None else

    return f"{'Intel Core ' if intel_match.group(2) is None else ''}{
elif amd_match:
    series = int(amd_match.group(3)[0]) * 1000
    return f"AMD Ryzen {amd_match.group(2)} {series} Series".strip()
else:
    return str(processor) # return original if no match
```

This is a function that drops the values that does not match the specific pattern

```
In [ ]: def processorName(name):
pattern = r"(Intel Core i[3579] \d{1,2}th Gen\.)|(AMD Ryzen [3579] \d"
return bool(re.match(pattern, str(name)))
```



```
In [ ]: ebay_df = ebay_df[ebay_df['Processor'].apply(processorName)]
dropped_val = ebay_df[ebay_df['Processor'] == "Intel Core i3 11th Gen., I
ebay_df.drop(dropped_val, inplace=True)
```

```
In [ ]: ebay_df.isnull().sum()
```

```
Out[ ]: Brand                0
Product_Description         0
Screen_Size                 0
RAM                         0
Processor                   0
GPU                        348
GPU_Type                    234
Resolution                  387
Condition                   0
Price                       0
dtype: int64
```

2.1.5 Conditions

I am removing the - refurbished str so that my visualisation can be cleaner

```
In [ ]: ebay_df['Condition'] = ebay_df['Condition'].str.replace('- Refurbished',
```

2.1.6 Dropping Na

I am omitting the Product description because it's unnecessary for my needs, and I'm skipping the GPU type since I can derive it from the specific GPU column.

Additionally, I'm excluding rows from the GPU and Resolution columns because they show little to no commonality with the other columns, so I've chosen not to include them.

```
In [ ]: ebay_df.drop('Product_Description', axis=1, inplace=True)
ebay_df.drop('GPU_Type', axis=1, inplace=True)
ebay_df.dropna(subset=['GPU', 'Resolution'], inplace=True)
```

2.1.7 Saving clean file to refer

```
In [ ]: ebay_df.to_csv('cleaned_ebay.csv', index=False)
```

2.2 Feature Engineering

i am making a new column so that i can group the price ranges

```
In [ ]: def categorize_price(price):
    if price < 300:
        return 'budget'
    elif price < 600:
        return 'mid-range'
    else:
```

```
        return 'high-end'

ebay_df['Price_Category'] = ebay_df['Price'].apply(categorize_price)
```

2.3 Exploratory Data Analysis

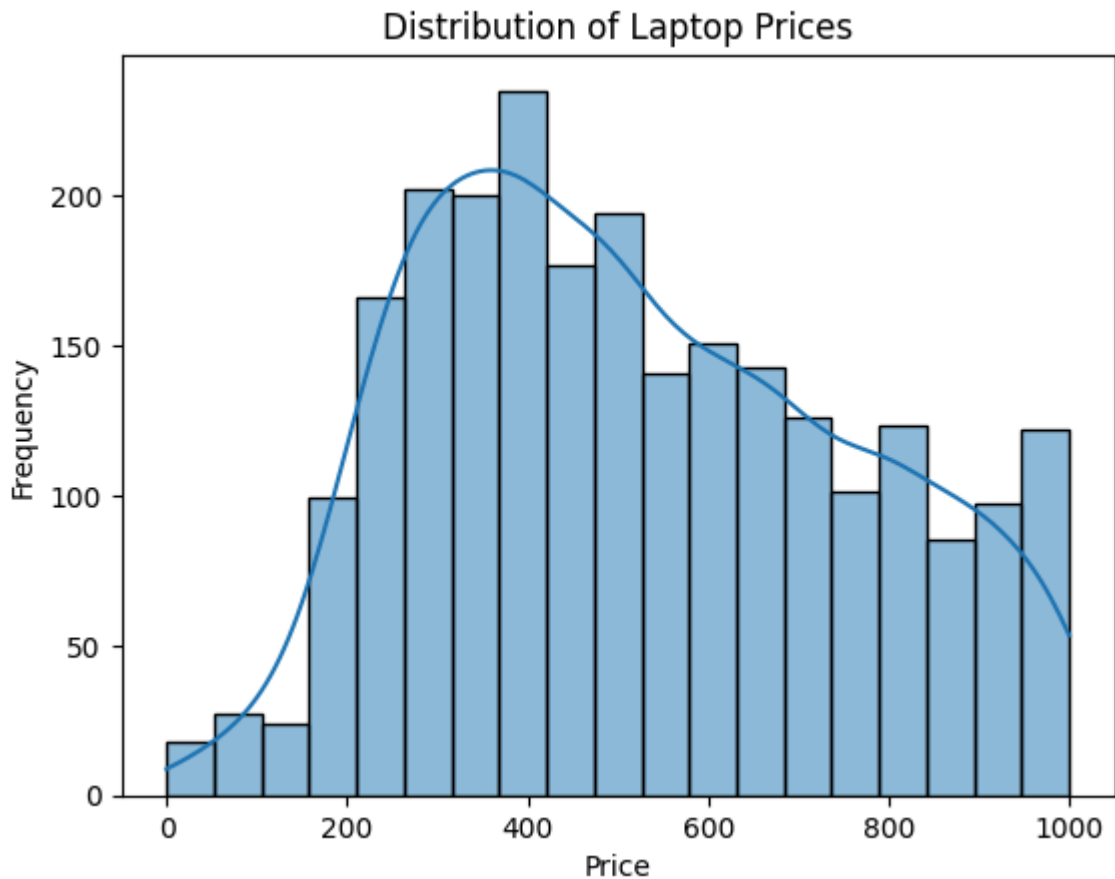
2.3.1 Popularity based on prices

The histogram shows the distribution of laptop prices. The x-axis represents the price range, and the y-axis represents the frequency of laptops in each price range. The histogram shows that the distribution of laptop prices is skewed to the right, meaning that there are more laptops priced lower than the average price than there are laptops priced higher than the average price. The KDE (Kernel Density Estimation) line shows the smoothed probability density function of the data.

- The majority of laptops are priced between 200 and 600.
- There is a peak in the frequency around \$400, indicating that this price range is most common.
- The frequency gradually decreases as the price increases beyond \$600.
- There are fewer laptops available at higher price points.

This suggests that the market for laptops is more competitive at lower price points, and there are more options available for consumers looking for affordable laptops. As the price increases, the market becomes more niche, with fewer options available.

```
In [ ]: # Distribution of prices
sns.histplot(ebay_df['Price'], kde=True)
plt.title('Distribution of Laptop Prices')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()
```



2.3.2 Prices based on Brand

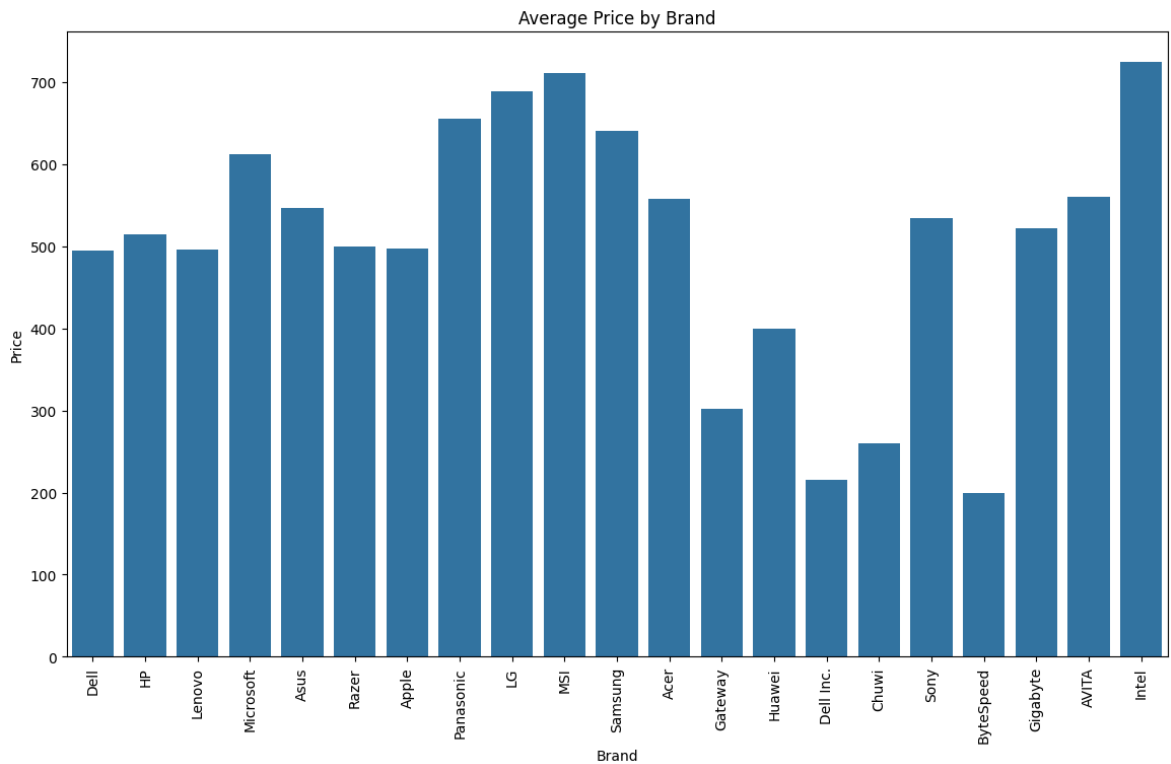
The bar chart shows the average price of laptops by brand. The x-axis lists the brands, and the y-axis shows the average price.

- Intel laptops are the most expensive, with an average price of over 700.
- MSI, LG, and Panasonic have the next highest average prices, all above 650.
- Microsoft, Lenovo, and HP have average prices in the 500-550 range.
- Acer, Gateway, and Huawei fall in the 300-400 range.
- Asus, Razer, and Apple have relatively similar average prices, around 500.

Overall, the chart shows a wide range of prices across different laptop brands. It suggests that consumers have a variety of choices based on their budget and needs.

```
In [ ]: plt.figure(figsize=(14, 8))
sns.barplot(x='Brand', y='Price', data=ebay_df, errorbar=None)
plt.xticks(rotation=90)
plt.title('Average Price by Brand')
```

```
Out[ ]: Text(0.5, 1.0, 'Average Price by Brand')
```

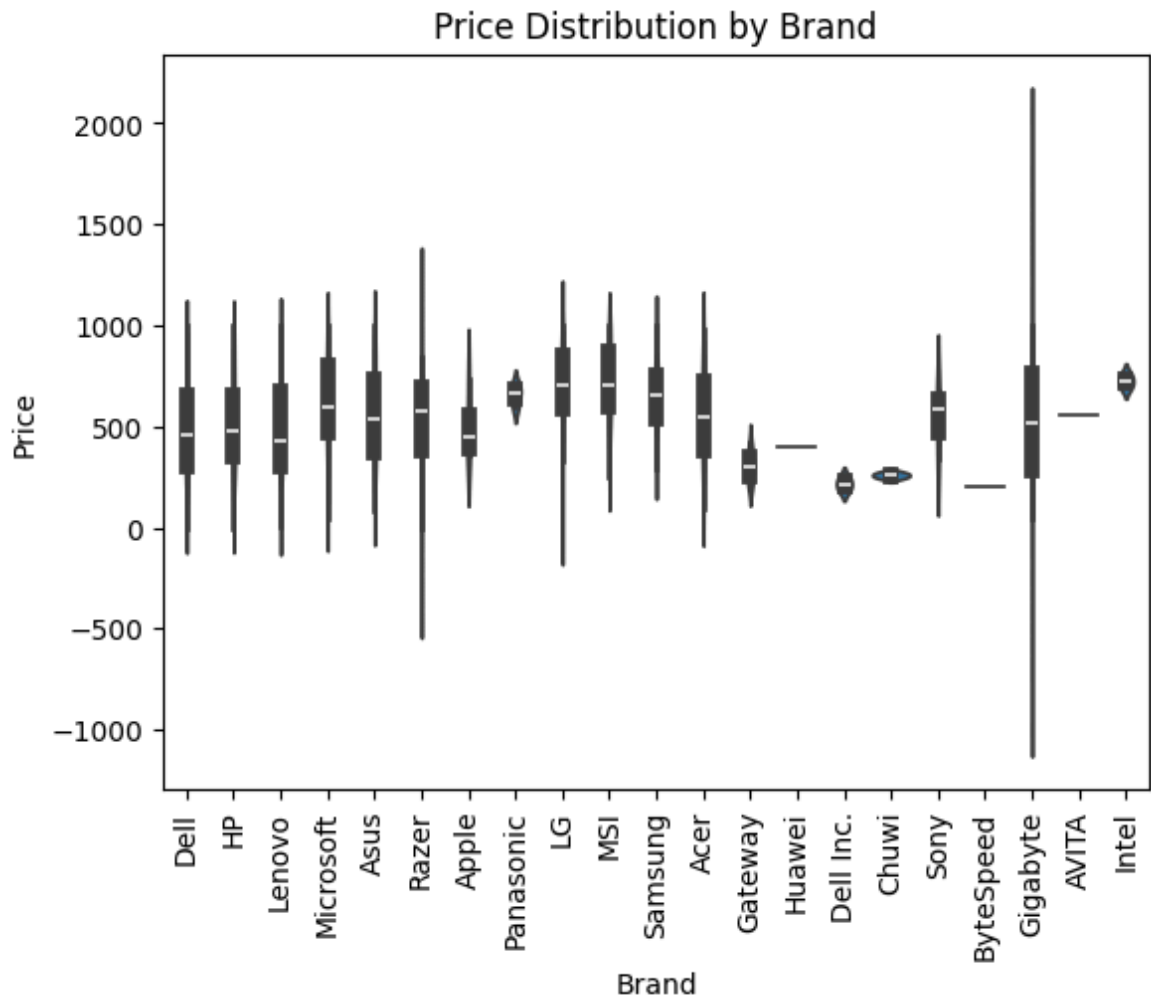


The boxplot shows the distribution of laptop prices by brand. It reveals several interesting observations:

- **Price Range:** The price range of laptops varies significantly across brands. Some brands, like Intel and Gigabyte, have very high prices, while others, like Chuwi and Dell Inc., tend to have lower prices.
- **Median Price:** The median price for most brands falls between 0 and 500, indicating that the majority of laptops within these brands are in the lower to mid-price range.
- **Outliers:** Some brands, like Intel and Gigabyte, have numerous outliers with very high prices, suggesting the presence of premium or high-performance models.
- **Distribution:** The boxplot also highlights the spread of prices within each brand. For instance, brands like Acer, Gateway, and Huawei have more tightly clustered price distributions, suggesting a more focused product offering. In contrast, brands like Dell and Lenovo have broader price distributions, suggesting a greater variety of models at different price points.

Overall, the boxplot helps visualize the price landscape of laptop brands and provides insights into their product positioning and pricing strategies.

```
In [ ]: # Violin plot of Price by Brand
sns.violinplot(x='Brand', y='Price', data=ebay_df)
plt.title('Price Distribution by Brand')
plt.xlabel('Brand')
plt.ylabel('Price')
plt.xticks(rotation=90)
plt.show()
```

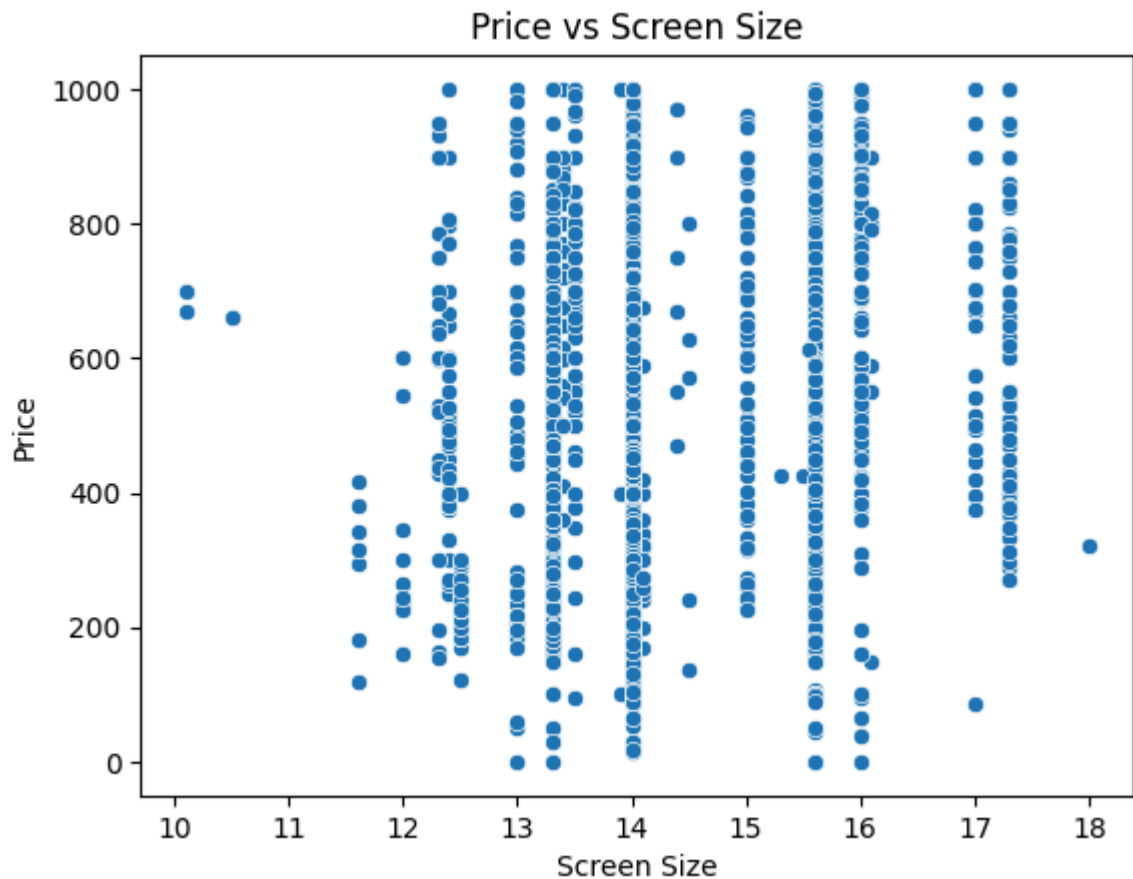


2.3.3 Price based on Screen size

this is a scatter plot showing the relationship between the price of a product and its screen size.

- **No Strong Correlation:** There doesn't appear to be a strong linear correlation between price and screen size. This means that screen size alone doesn't directly dictate the price of a product.
- **Clustering:** There are clusters of data points, suggesting that certain screen sizes are more common. The most prominent clusters are around the screen sizes of 15.6, 14, 13, and 12 inches.
- **Price Range:** The price range of the products appears to be fairly wide, spanning from around 0 to 1000 (assuming the y-axis represents price).
- **Possible Outliers:** There are a few data points that seem to be far away from the main clusters. These might be products with unique features that justify their higher price despite having a smaller screen size.

```
In [ ]: sns.scatterplot(x='Screen_Size', y='Price', data=ebay_df)
plt.title('Price vs Screen Size')
plt.xlabel('Screen Size')
plt.ylabel('Price')
plt.show()
```



2.3.4 Price based on Processor

The graph shows the average price of laptops based on the CPU model. The higher the bar, the higher the average price. The graph shows that laptops with the latest generation CPUs tend to have higher average prices. For example, the average price of laptops with AMD Ryzen 7 6000 Series CPUs is higher than the average price of laptops with AMD Ryzen 5 7000 Series CPUs. This is likely because the latest generation CPUs offer better performance and features.

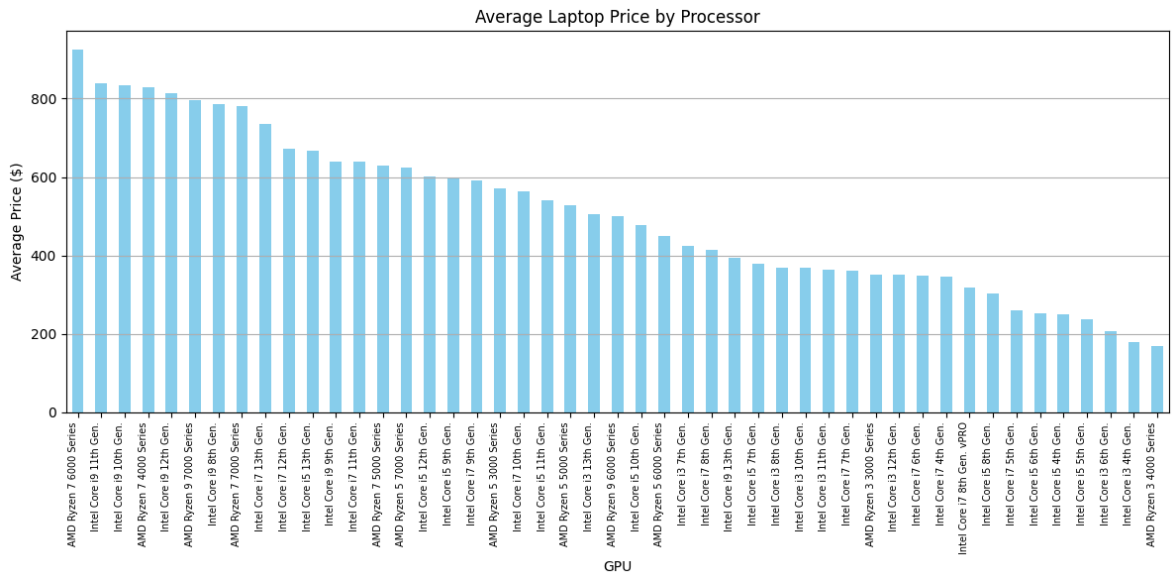
In addition, the graph shows that laptops with Intel CPUs generally have higher average prices than laptops with AMD CPUs. This is likely because Intel CPUs are typically more powerful than AMD CPUs, especially for gaming and other demanding tasks.

Overall, the graph shows that there is a strong correlation between the generation of the CPU and the average price of a laptop. The latest generation CPUs tend to have higher average prices, as they offer better performance and features.

```
In [ ]: average_price_per_gpu = ebay_df.groupby('Processor')['Price'].mean().sort

# Plotting
plt.figure(figsize=(12, 6))
average_price_per_gpu.plot(kind='bar', color='skyblue')
plt.title('Average Laptop Price by Processor')
plt.xlabel('GPU')
plt.ylabel('Average Price ($)')
plt.xticks(rotation=90, ha='right', fontsize=7)
```

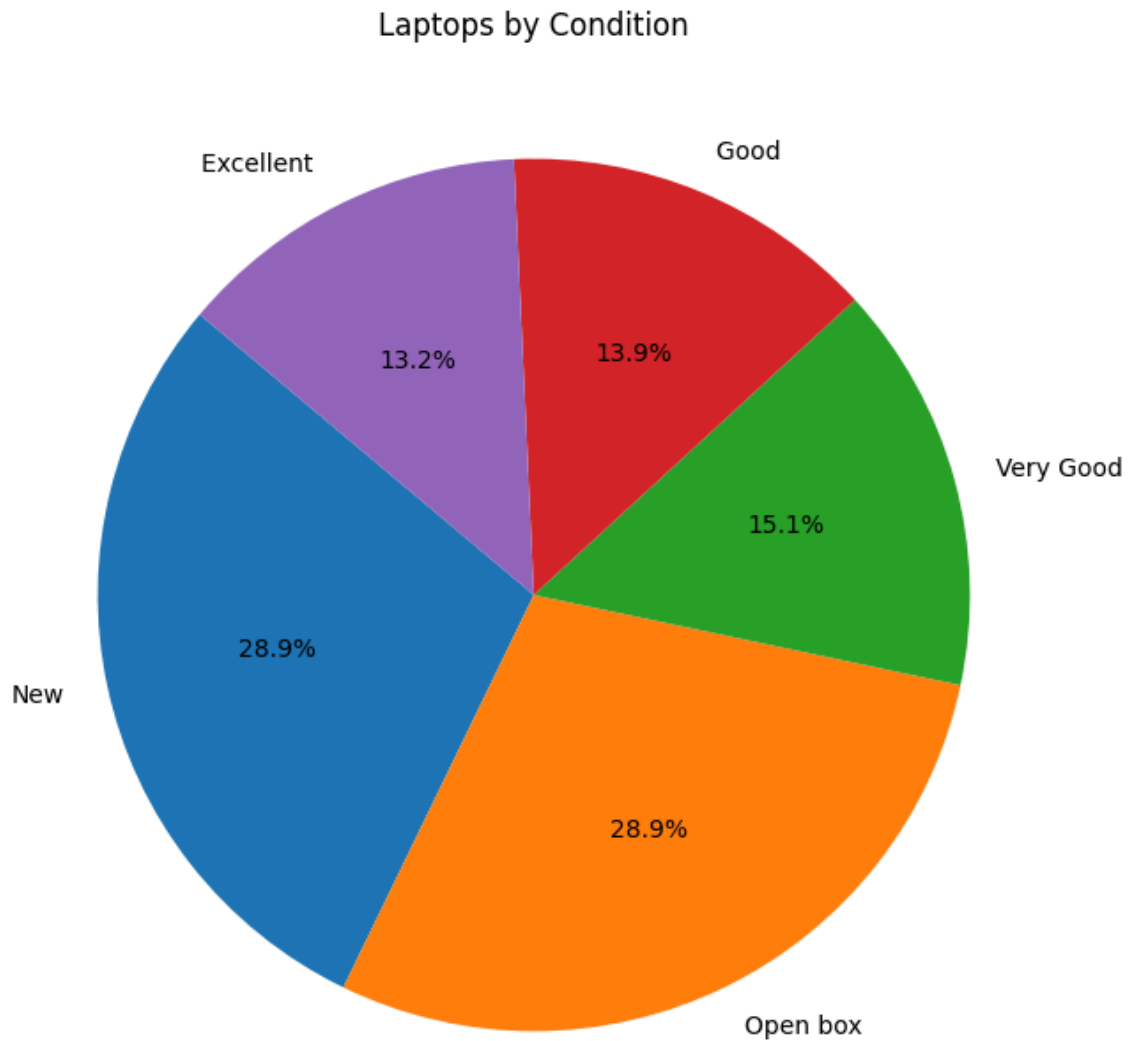
```
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```



2.3.5 Popularity based on Conditions

The pie chart shows the distribution of laptops by condition. The largest proportion of laptops are in "New" condition and "Open box" condition, each accounting for 28.9%. This is followed by "Very Good" (15.1%), "Good" (13.9%), and "Excellent" (13.2%). This suggests that there is a high demand for new and open box laptops, likely due to their lower prices and potential warranty benefits. The smaller proportion of laptops in the "Excellent" and "Good" conditions indicates that there are fewer customers looking for used laptops, possibly due to concerns about their quality and reliability.

```
In [ ]: condition_counts = ebay_df['Condition'].value_counts()
plt.figure(figsize=(8, 8))
plt.pie(condition_counts, labels=condition_counts.index, autopct='%1.1f%%')
plt.title('Laptops by Condition')
plt.show()
```



3 Amazon

3.1 Data Cleaning and Processing

I am loading and exploring the Amazon csv file using the pandas library. i first examine the shape and the summary on the dataframe to understand what has to be done to clean it at a glance

```
In [ ]: amazon_df = pd.read_csv('Amazon.csv')
```

```
In [ ]: amazon_df.info()
```



```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4446 entries, 0 to 4445
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   brand                 4446 non-null   object
1   model                 3282 non-null   object
2   screen_size           4417 non-null   object
3   color                 3867 non-null   object
4   harddisk              3870 non-null   object
5   cpu                   4346 non-null   object
6   ram                   4385 non-null   object
7   OS                    4420 non-null   object
8   special_features      2054 non-null   object
9   graphics              4381 non-null   object
10  graphics_coprocessor  2574 non-null   object
11  cpu_speed              1520 non-null   object
12  rating                 2174 non-null   float64
13  price                 4441 non-null   object
dtypes: float64(1), object(13)
memory usage: 486.4+ KB

```

3.1.1 Screen Size

I am replacing the word Inches to an empty space so that the code can be shortened and it is cleaner if i want to display it on a graph

```
In [ ]: amazon_df['screen_size'] = amazon_df['screen_size'].str.replace(" Inches"
```

```
In [ ]: amazon_df.screen_size = pd.to_numeric(amazon_df.screen_size)
amazon_df.screen_size
```

```

Out[ ]: 0      14.00
1      15.60
2      15.66
3      13.30
4      15.30
...
4441   15.60
4442   14.00
4443   15.60
4444   14.00
4445   14.00
Name: screen_size, Length: 4446, dtype: float64

```

2.1.2 RAM

I am checking and removing the GB and MB values as i assume that the MB is wrongly placed and removing the words allows me to change the value of the ram columns to a float value

```
In [ ]: amazon_df.ram
```

```
Out[ ]: 0      8 GB
        1     64 GB
        2     32 GB
        3      8 GB
        4      8 GB
        ...
        4441    32 GB
        4442      8 GB
        4443      8 GB
        4444      8 GB
        4445      NaN
        Name: ram, Length: 4446, dtype: object
```

```
In [ ]: amazon_df.ram.replace(" GB","", inplace=True, regex=True)
        amazon_df.ram.replace(" MB","", inplace=True, regex=True)
```

```
In [ ]: amazon_df.ram = pd.to_numeric(amazon_df.ram)
        amazon_df.ram
```

```
Out[ ]: 0      8.0
        1     64.0
        2     32.0
        3      8.0
        4      8.0
        ...
        4441    32.0
        4442      8.0
        4443      8.0
        4444      8.0
        4445      NaN
        Name: ram, Length: 4446, dtype: float64
```

3.1.3 Price

```
In [ ]: print(amazon_df.price)
```

```
0      $589.99
1      $999.99
2     $1,599.00
3      $689.99
4     $1,144.48
        ...
4441    $1,699.00
4442      $589.99
4443      $591.99
4444    $1,771.99
4445      $938.99
        Name: price, Length: 4446, dtype: object
```

i am just replacing the dollar sign and the comma in the price section of the data

```
In [ ]: amazon_df.price = amazon_df.price.str.replace('$', '')
        amazon_df.price = amazon_df.price.str.replace(',', '')
        amazon_df.price.value_counts()
        amazon_df['price'] = amazon_df['price'].astype(float)
```

3.1.4 Brand

i am doing the same thing as how i cleaned the eBay function. i am manually checking and replacing the name so that the code would be able to understand the systematic approach and would be able to group them

```
In [ ]: amazon_df['brand'] = amazon_df['brand'].replace('ASUS', 'Asus')
amazon_df['brand'] = amazon_df['brand'].replace('AWOW', 'Awow')
amazon_df['brand'] = amazon_df['brand'].replace('Alienware', 'Dell')
amazon_df['brand'] = amazon_df['brand'].replace('CTL', 'Ctl')
amazon_df['brand'] = amazon_df['brand'].replace('DELL', 'Dell')
amazon_df['brand'] = amazon_df['brand'].replace('IVIEW', 'Iview')
amazon_df['brand'] = amazon_df['brand'].replace('JTD', 'Jtd')
amazon_df['brand'] = amazon_df['brand'].replace('LENOVO', 'Lenovo')
amazon_df['brand'] = amazon_df['brand'].replace('MAINGEAR', 'Maingear')
amazon_df['brand'] = amazon_df['brand'].replace('MICROTELLA', 'Microtella')
amazon_df['brand'] = amazon_df['brand'].replace('SAMSUNG', 'Samsung')
amazon_df['brand'] = amazon_df['brand'].replace('SHOXLAB', 'Shoxlab')
amazon_df['brand'] = amazon_df['brand'].replace('TOPOSH', 'Toposh')
amazon_df['brand'] = amazon_df['brand'].replace('VAIO', 'Sony')
amazon_df['brand'] = amazon_df['brand'].replace('XAMMUE', 'Xammue')
amazon_df['brand'] = amazon_df['brand'].replace('acer', 'Acer')
amazon_df['brand'] = amazon_df['brand'].replace('asus', 'Asus')
amazon_df['brand'] = amazon_df['brand'].replace('dell', 'Dell')
amazon_df['brand'] = amazon_df['brand'].replace('hp', 'HP')
amazon_df['brand'] = amazon_df['brand'].replace('lenovo', 'Lenovo')
```

3.1.5 Processors

Similar to what i did for the eBay Processors, i did another version which is less complicated but more tedious which changes a specific word to the name of the CPU

```
In [ ]: def transformar_cpu(core):
    core = 'Intel Core i3' if 'i3' in core.lower() else core
    core = 'Intel Core i5' if 'i5' in core.lower() else core
    core = 'Intel Core i9' if 'i9' in core.lower() else core
    core = 'Intel Core i7' if 'i7' in core.lower() else core
    core = 'Ryzen 3' if 'Ryzen 3' in core.lower() else core
    core = 'Ryzen 5' if 'Ryzen 5' in core.lower() else core
    core = 'Ryzen 7' if 'Ryzen 7' in core.lower() else core
    core = 'Ryzen 9' if 'Ryzen 9' in core.lower() else core
    core = 'Celeron' if 'celeron' in core.lower() else core
    core = 'Pentium' if 'pentium' in core.lower() else core
    return core

amazon_df['cpu'] = amazon_df['cpu'].fillna('')
amazon_df['cpu'] = amazon_df['cpu'].apply(transformar_cpu)
```

I have created a function that specifically focuses on detecting Ryzen and Intel processors, as these two brands are the most prevalent and widely used in laptops, dominating the market share.

```
In [ ]: def simplify_processor_name(name):
    ryzen_pattern = re.compile(r'(Ryzen\s\d+)')
    ryzen_match = ryzen_pattern.search(name)
    if ryzen_match:
        return ryzen_match.group(1)
```

```

intel_pattern = re.compile(r'(Intel\s(Core\s(i\d+|Duo|2\sDuo|2\sQuad|
intel_match = intel_pattern.search(name)
if intel_match:
    return intel_match.group(1)

return "Other"

amazon_df['cpu'] = amazon_df['cpu'].apply(simplify_processor_name)
amazon_df = amazon_df[amazon_df['cpu'] != "Other"]
print(amazon_df['cpu'].unique())

```

```

['Intel Core i7' 'Intel Core i5' 'Intel Core i9' 'Ryzen 3' 'Intel Core i3'
'Ryzen 7' 'Ryzen 5' 'Ryzen 9' 'Intel Atom' 'Intel Core Duo'
'Intel Core 2 Quad']

```

3.1.6 Dropping Na and renaming

checking which column and NA and making sure i do not need the column before dropping it. i am also renaming the columns so that it matches the eBay column to make referencing and visualisation easier

```

In [ ]: amazon_df = amazon_df.rename(columns={'cpu': 'Processor'})
amazon_df = amazon_df.rename(columns={'screen_size': 'Screen_Size'})
amazon_df = amazon_df.rename(columns={'price': 'Price'})

```

I am intentionally dropping these columns to ensure a seamless merge of the two dataframes. By removing these columns, I can adapt the data sheet to facilitate a successful consolidation of the data

```

In [ ]: amazon_df.drop('model', axis=1, inplace=True)
amazon_df.drop('color', axis=1, inplace=True)
amazon_df.drop('harddisk', axis=1, inplace=True)
amazon_df.drop('OS', axis=1, inplace=True)
amazon_df.drop('special_features', axis=1, inplace=True)
amazon_df.drop('graphics_coprocessor', axis=1, inplace=True)
amazon_df.drop('cpu_speed', axis=1, inplace=True)
amazon_df.drop('graphics', axis=1, inplace=True)

```

```

In [ ]: amazon_df.dropna(subset=['rating'], inplace=True)

```

```

In [ ]: amazon_df.to_csv('cleaned_Amazon.csv', index=False)

```

3.2 Feature Engineering

i made two more columns based on the ratings and price to determine the customer satisfaction and the price range to categorise them more accurately

```

In [ ]: def categorize_rating(rating):
    if rating < 3.5:
        return 'neutral'
    elif rating < 4:
        return 'satisfied'
    else:
        return 'very_satisfied'

```

```
amazon_df['Satisfaction'] = amazon_df['rating'].apply(categorize_rating)

def categorize_price(price):
    if price < 300:
        return 'budget'
    elif price < 600:
        return 'mid-range'
    else:
        return 'high-end'

amazon_df['Price_Category'] = amazon_df['Price'].apply(categorize_price)
```

3.3 Exploratory Data Analysis

3.3.1 Popularity based on Prices

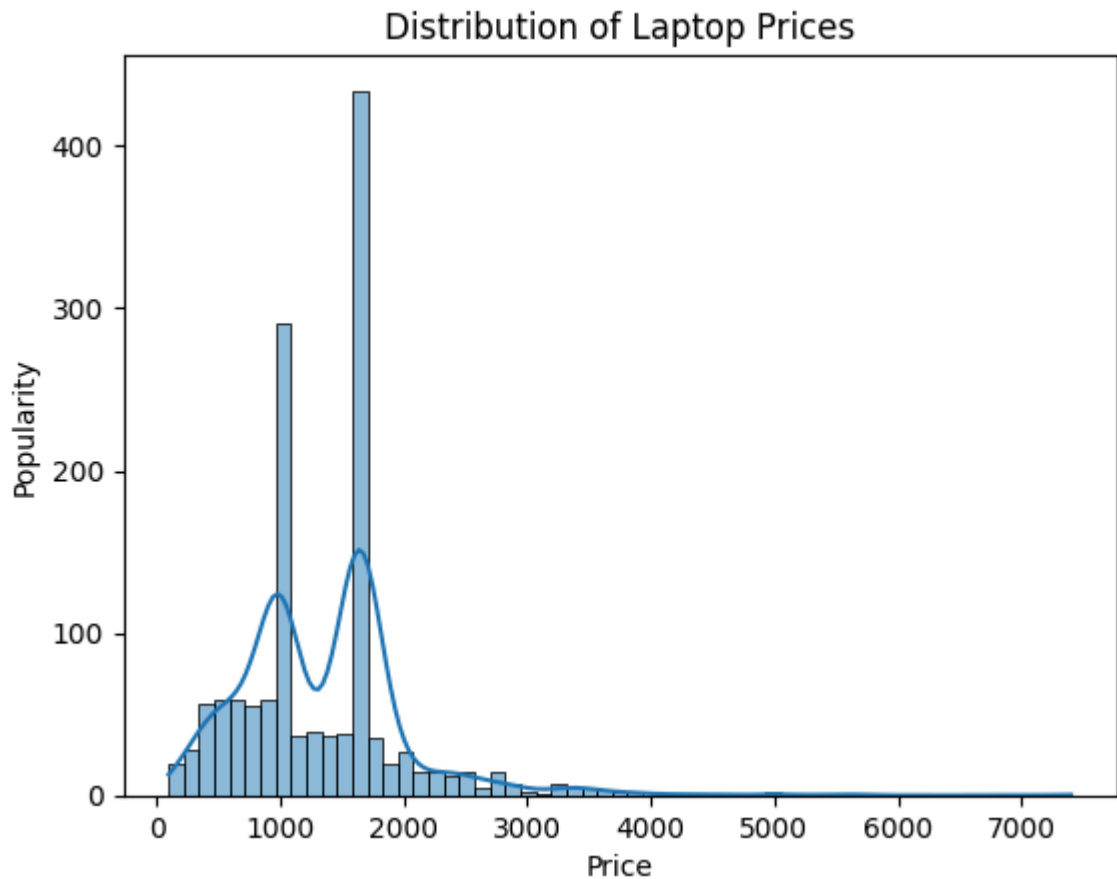
The graph shows the distribution of laptop prices. The x-axis represents the price of the laptops, and the y-axis represents the frequency of laptops at that price. The graph shows that the distribution of laptop prices is skewed to the right, meaning that there are more laptops with lower prices than there are laptops with higher prices. There are two clear peaks in the graph, one around 1000 and another around 1700. This suggests that there are two price points where laptops are most popular.

The peak around

1700 might represent the price range for mid-range laptops, while the peak around 1000 could represent more budget-friendly options. It's also worth noting that there are fewer laptops with prices above 3000, showing that high-end laptops are less common than lower-priced options.



```
In [ ]: # Distribution of prices
sns.histplot(amazon_df['Price'], kde=True)
plt.title('Distribution of Laptop Prices')
plt.xlabel('Price')
plt.ylabel('Popularity')
plt.show()
```

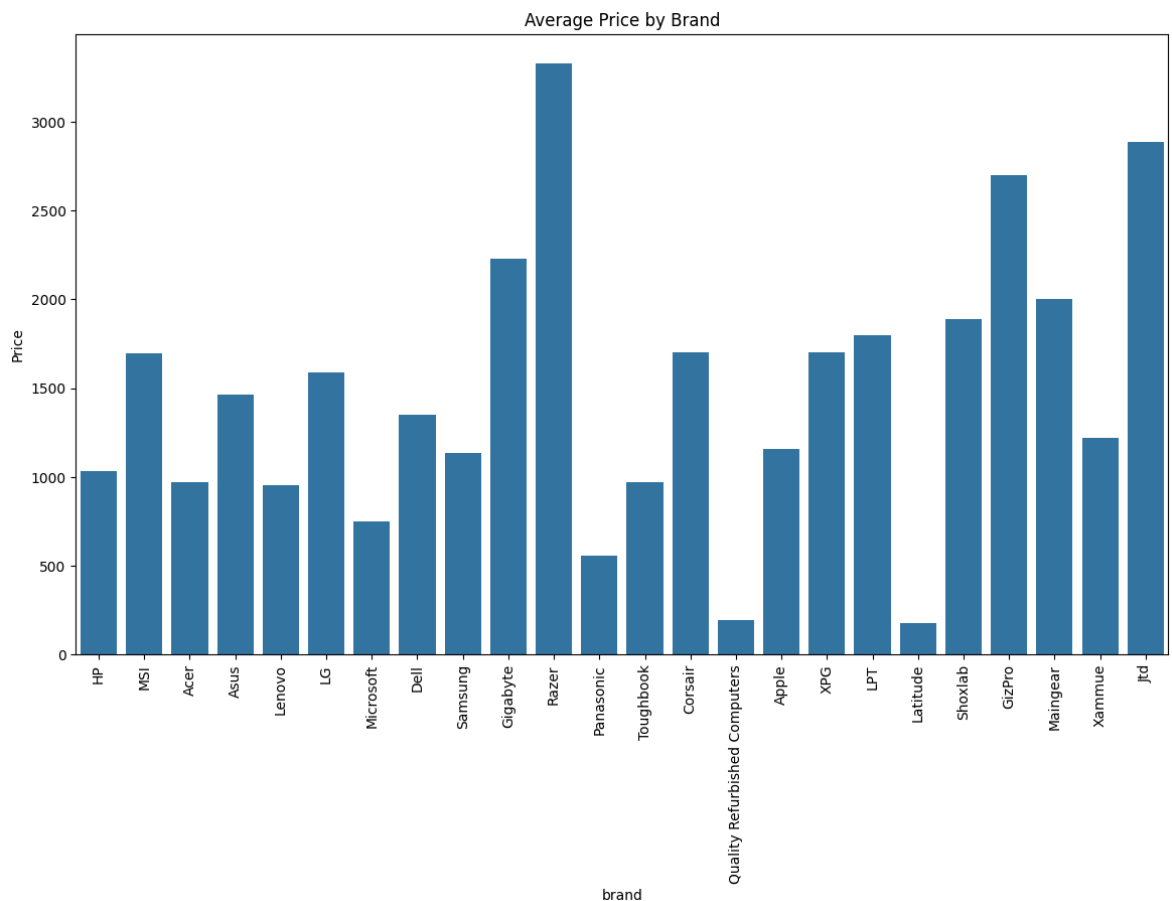


3.3.2 Price based on Processor

The graph shows the average price of laptops by brand. The brand with the highest average price is Razer, followed by Gigabyte and Jtd. The brand with the lowest average price is Corsair, followed by Latitude and Shoxlab. It is important to note that this is just a general overview of laptop prices by brand, and the actual price of a specific laptop will vary depending on its specific features and specifications.

```
In [ ]: plt.figure(figsize=(14, 8))
sns.barplot(x='brand', y='Price', data=amazon_df, errorbar=None)
plt.xticks(rotation=90)
plt.title('Average Price by Brand')
```

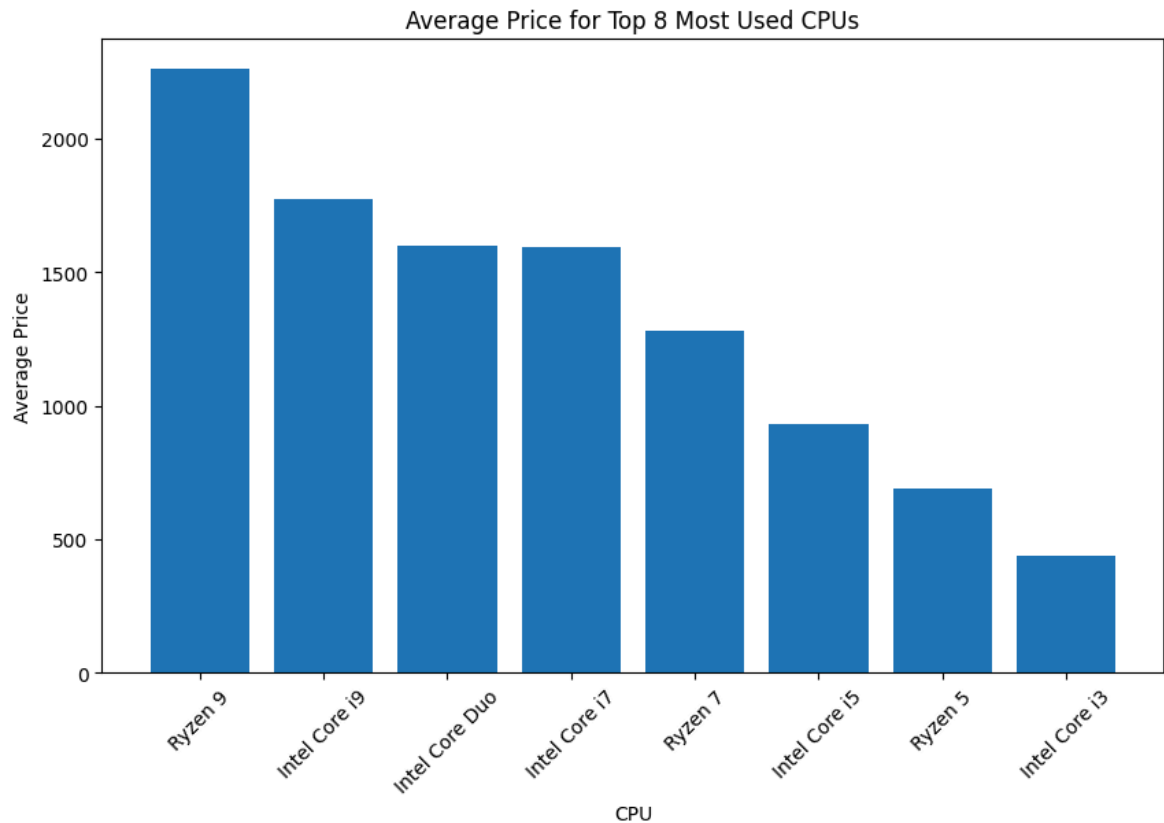
```
Out[ ]: Text(0.5, 1.0, 'Average Price by Brand')
```



3.3.3 Price based on Processor

The bar chart displays the average price of the top 8 most used CPUs. The CPUs are ranked from highest to lowest average price. The Ryzen 9 has the highest average price, followed by Intel Core i9 and Intel Core Duo. The Ryzen 7 and Intel Core i5 are in the middle, with the Ryzen 5 and Intel Core i3 at the bottom. This chart suggests that the more powerful and advanced CPUs, generally those with more cores and higher clock speeds, tend to be more expensive.

```
In [ ]: grouped = amazon_df.groupby('Processor')['Price'].mean().nlargest(8)
plt.figure(figsize=(10, 6))
plt.bar(grouped.index, grouped.values)
plt.xlabel('CPU')
plt.ylabel('Average Price')
plt.title('Average Price for Top 8 Most Used CPUs')
plt.xticks(rotation=45)
plt.show()
```



3.3.4 Heat map overview

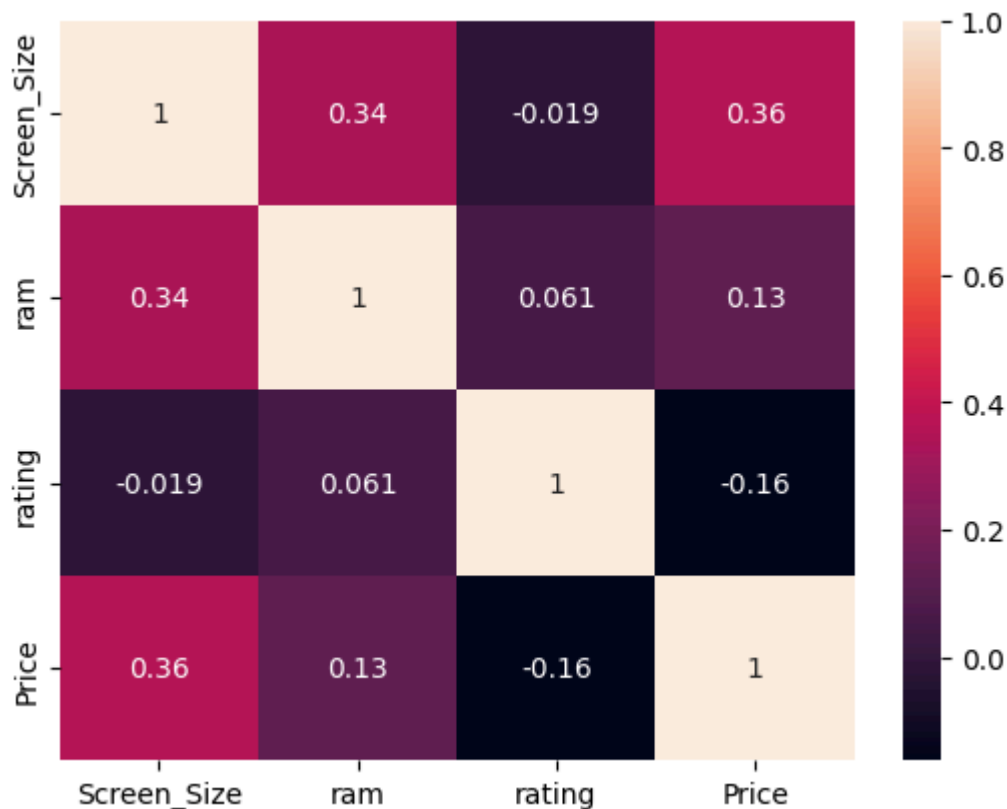
This is a heatmap showing the correlation between different features of a mobile phone dataset. The features are Screen Size, RAM, rating and Price. A correlation of 1 indicates a perfect positive relationship, a correlation of 0 indicates no relationship and a correlation of -1 indicates a perfect negative relationship.

The heatmap shows a moderate positive correlation between Screen Size and Price (0.36), indicating that larger screen sizes tend to have higher prices. There is also a moderate positive correlation between RAM and Price (0.34). This makes sense as larger RAM capacity is typically associated with more expensive devices.

The correlation between rating and Price is very weak (-0.16), suggesting that there is no strong relationship between customer ratings and the price of a phone. This suggests that price is not the only factor driving customer satisfaction with mobile phones.

Overall, the heatmap provides insights into the relationships between different features of mobile phones, which can be useful for understanding pricing strategies, customer preferences, and product development.

```
In [ ]: plt.figure()  
sns.heatmap(amazon_df.select_dtypes(exclude=object).corr(), annot=True);
```

4 Market analysis

this part is analysis both ebay and amazon data set to make a more informed decision

```
In [ ]: def transformar_cpu(core):
    core = 'Intel Core i3' if 'i3' in core.lower() else core
    core = 'Intel Core i5' if 'i5' in core.lower() else core
    core = 'Intel Core i9' if 'i9' in core.lower() else core
    core = 'Intel Core i7' if 'i7' in core.lower() else core
    core = 'Ryzen 3' if 'ryzen 3' in core.lower() else core
    core = 'Ryzen 5' if 'ryzen 5' in core.lower() else core
    core = 'Ryzen 7' if 'ryzen 7' in core.lower() else core
    core = 'Ryzen 9' if 'ryzen 9' in core.lower() else core
    return core

ebay_df['Processor'] = ebay_df['Processor'].fillna('')
ebay_df['Processor'] = ebay_df['Processor'].apply(transformar_cpu)
amazon_df['ecommerce_site'] = 'Amazon'
ebay_df['ecommerce_site'] = 'eBay'
amazon_df['Processor'].unique()
```

```
Out[ ]: array(['Intel Core i5', 'Intel Core i9', 'Ryzen 3', 'Intel Core i7',
               'Intel Core i3', 'Ryzen 7', 'Ryzen 5', 'Ryzen 9', 'Intel Atom',
               'Intel Core Duo'], dtype=object)
```

```
In [ ]: combined_df = pd.concat([ebay_df, amazon_df], ignore_index=True)
```

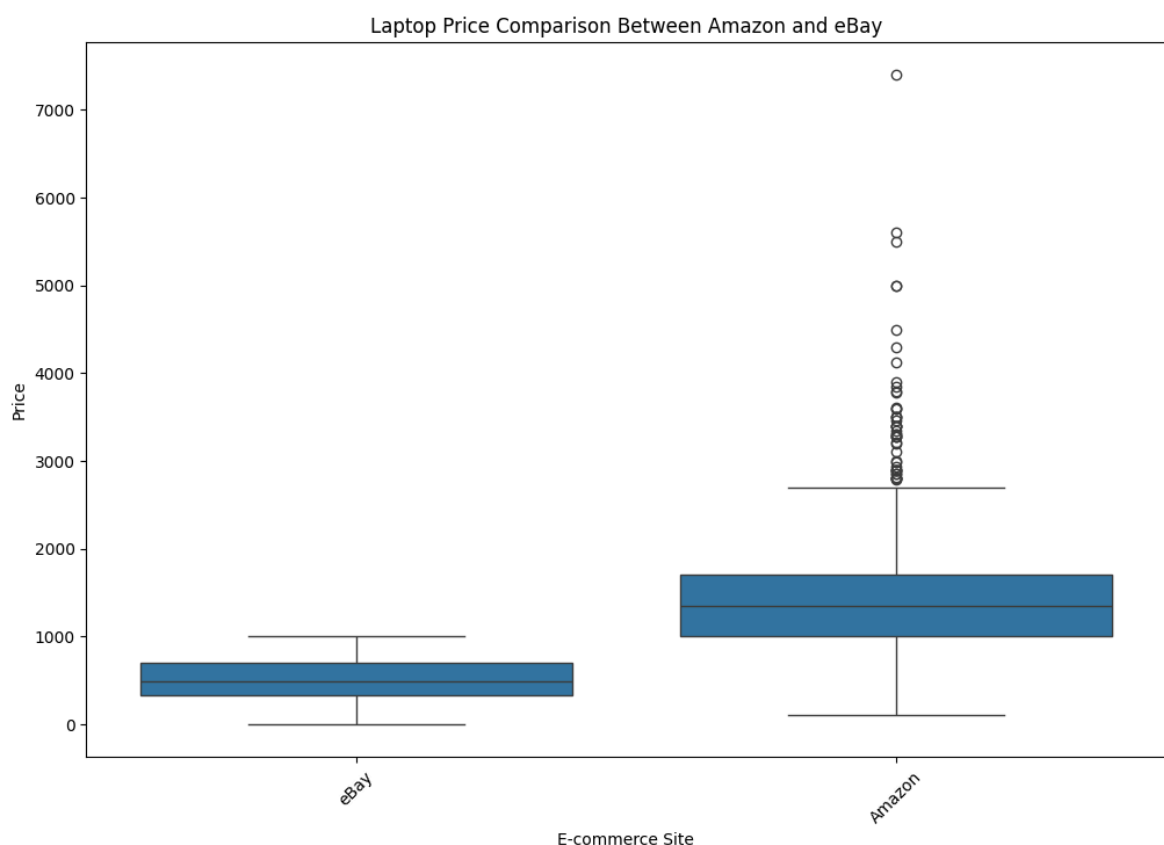
4.1 Amazon and Ebay Price comparison

The graph shows that Amazon prices tend to be higher, with a median price around 1500. eBay prices are more spread out, but generally lower, with a median price around 500. The whiskers of the box plots show the minimum and maximum values, while the box itself represents the interquartile range, which is the middle 50% of the data. The outliers are shown as individual points beyond the whiskers.

This suggests that for laptops, you might find better deals on eBay than Amazon, but you are also more likely to find outliers with extremely high or low prices on eBay. Amazon is more consistent in its pricing, but prices are generally higher.

```
In [ ]: # Step 2: Visualization
plt.figure(figsize=(12, 8))
sns.boxplot(x='ecommerce_site', y='Price', data=combined_df)
plt.title('Laptop Price Comparison Between Amazon and eBay')
plt.xlabel('E-commerce Site')
plt.ylabel('Price')
plt.xticks(rotation=45)

plt.show()
```



4.2 Correlation between Price and Processor

The first chart shows the average laptop price by processor (Intel and Ryzen). The second chart shows the processor popularity, which is the number of laptops with each processor.

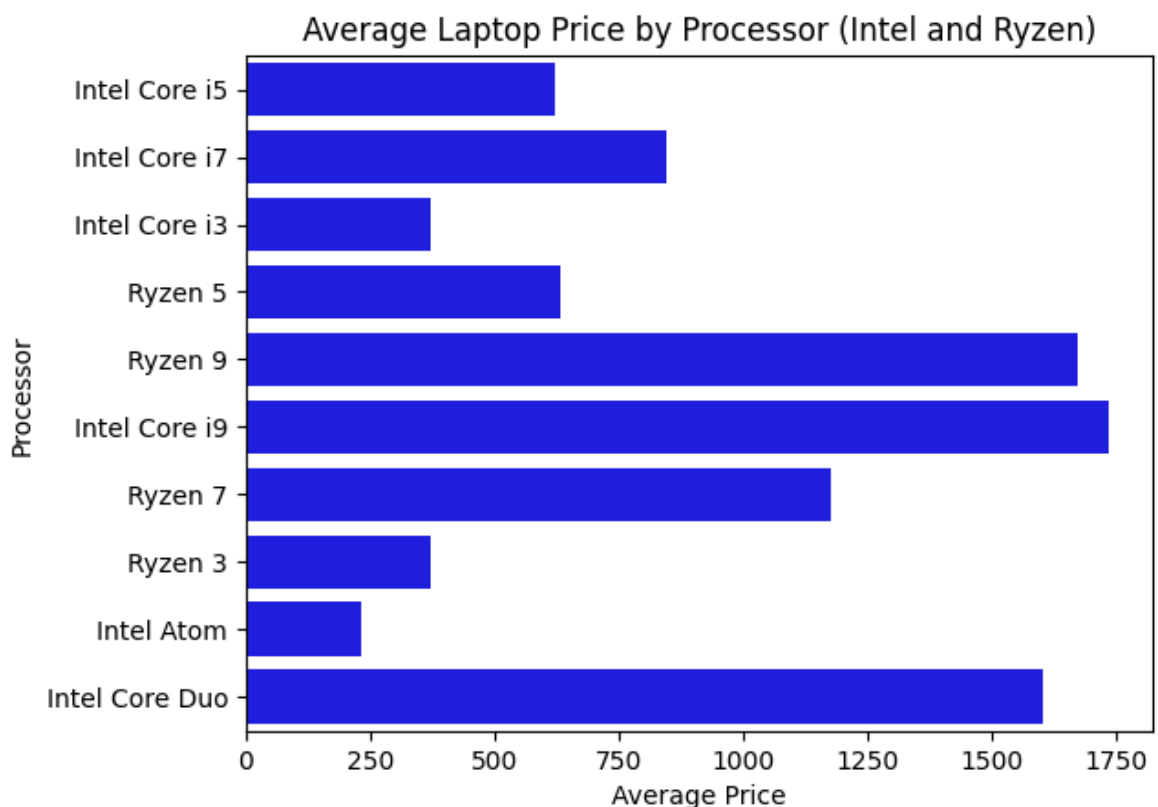
We can see that the more expensive processors are less popular. For example, the Ryzen 9 processor has the highest average price, but it is the least popular

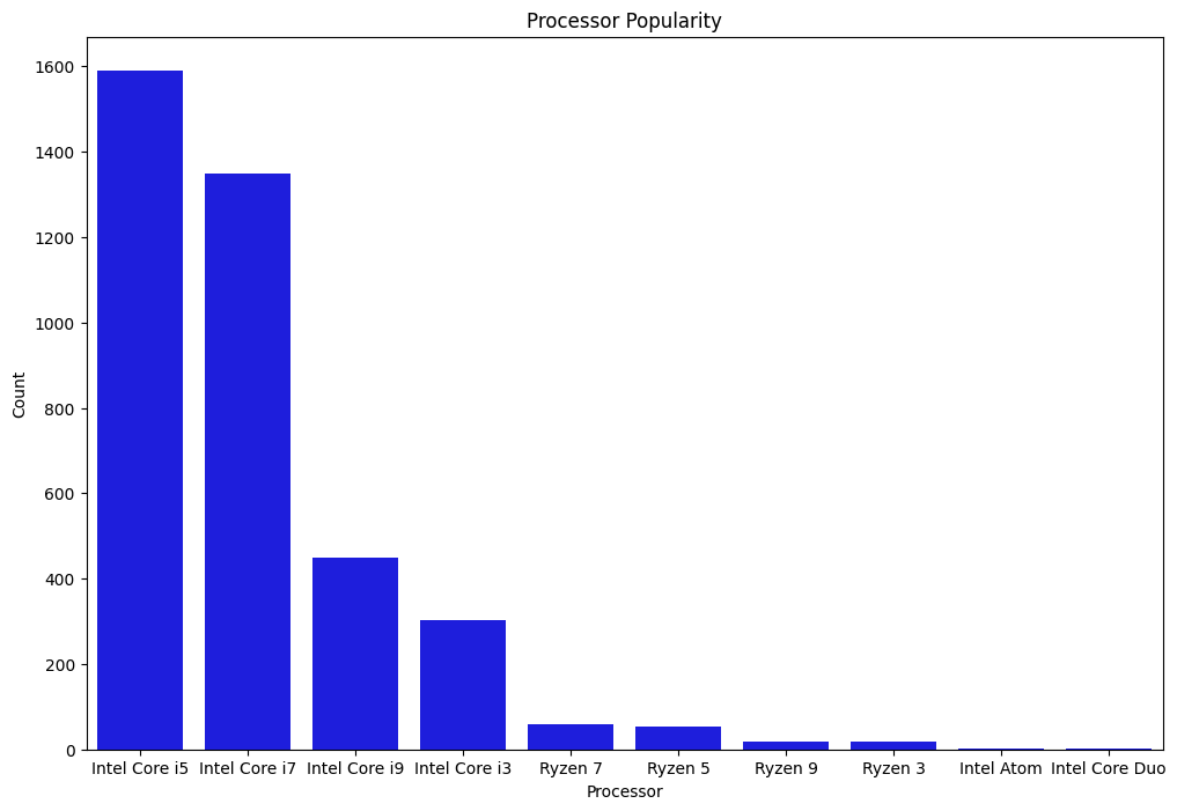
processor. This suggests that people are willing to pay more for higher performance processors, but they are not as common. On the other hand, the Intel Core i5 processor is the most popular processor, and it has a relatively low average price. This suggests that people are looking for a good balance between price and performance.

In summary, the first chart shows the relationship between processor price and performance, while the second chart shows the relationship between processor popularity and price. These two charts can be used to understand the market for laptops and to make informed decisions about which processor to buy.

```
In [ ]: sns.barplot(x='Price', y='Processor', data=combined_df, errorbar=None, co
plt.title('Average Laptop Price by Processor (Intel and Ryzen)')
plt.xlabel('Average Price')
plt.ylabel('Processor')
plt.show()

Processor_popularity = combined_df['Processor'].value_counts()
plt.figure(figsize=(12, 8))
sns.barplot(x=Processor_popularity.index, y=Processor_popularity.values,
plt.title('Processor Popularity')
plt.xlabel('Processor')
plt.ylabel('Count')
plt.show()
```





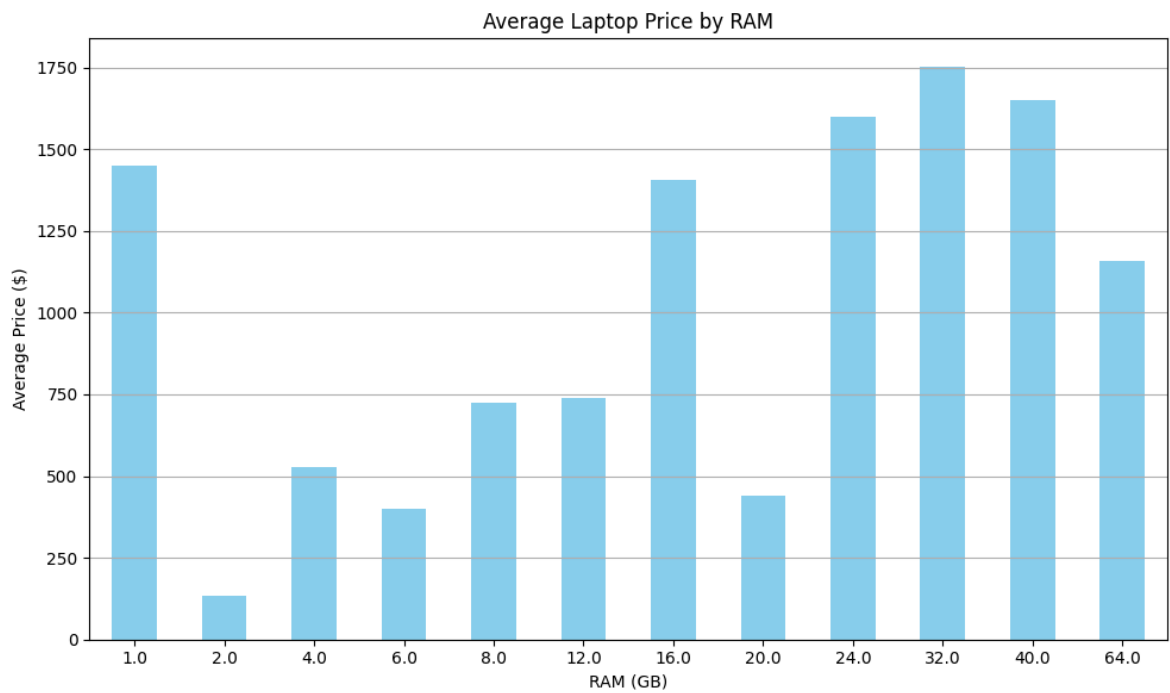
4.3 correlation between Price and Ram

The first graph shows the average laptop price by RAM. We can see that the average price of a laptop increases with the amount of RAM. The highest average price is for laptops with 32GB of RAM, followed by laptops with 40GB of RAM.

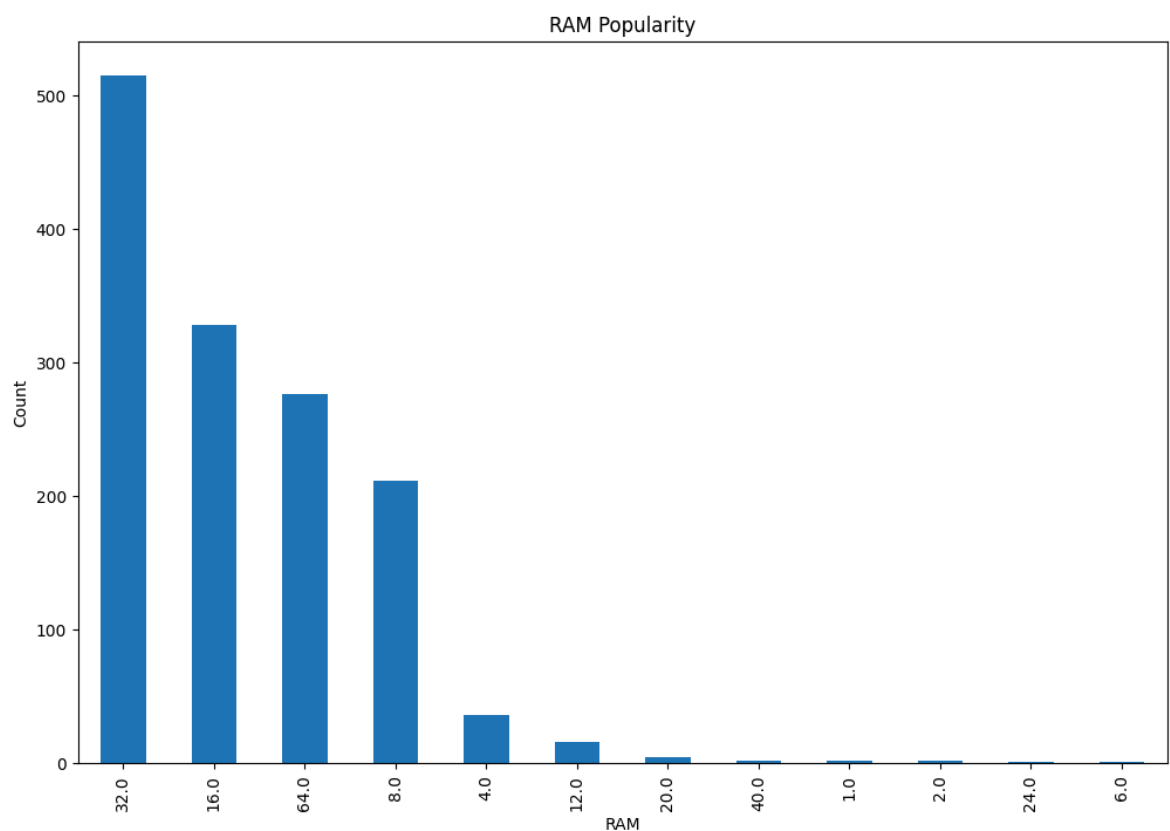
The second graph shows the popularity of different RAM sizes. We can see that the most popular RAM size is 32GB, followed by 16GB and 64GB. The least popular RAM sizes are 20GB, 1GB, and 2GB.

These two graphs together give us a good understanding of the relationship between RAM size and laptop price. We can see that while higher RAM sizes are more expensive, they are also more popular. This suggests that consumers are willing to pay more for higher RAM sizes, likely because they offer better performance.

```
In [ ]: avg_price_per_ram = combined_df.groupby('ram')['Price'].mean()
plt.figure(figsize=(10, 6))
avg_price_per_ram.plot(kind='bar', color='skyblue')
plt.title('Average Laptop Price by RAM')
plt.xlabel('RAM (GB)')
plt.ylabel('Average Price ($)')
plt.xticks(rotation=0)
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```



```
In [ ]: ram_popularity = combined_df['ram'].value_counts()
plt.figure(figsize=(12, 8))
ram_popularity.plot(kind='bar')
plt.title('RAM Popularity')
plt.xlabel('RAM')
plt.ylabel('Count')
plt.show()
```



4.4 Summary

In examining the data from the internet, we can gain insights into optimizing laptop components for cost savings, finding the most affordable prices, and identifying

popular laptop models.

Firstly, by analyzing the average price per processor, we can identify the processors that offer the best balance between performance and cost. The data shows that Intel Core i5 processors have a relatively low average price, making them a cost-effective choice for users who require decent performance without breaking the bank. Additionally, Ryzen 5 processors also offer good value for money, providing a viable alternative for budget-conscious consumers.

Secondly, the data reveals that eBay offers more affordable prices compared to Amazon. The box plot comparison between the two e-commerce sites shows that while Amazon prices tend to be higher and more consistent, eBay prices are more varied and generally lower. This suggests that users looking for cost savings should consider exploring eBay for potential deals on laptops. This is primarily because most of the laptops sold on eBay is second-hand which brings down the total average cost.

Thirdly, the popularity of laptop components can guide us in identifying the most sought-after laptop models. The processor popularity chart highlights that Intel Core i5 and Intel Core i7 processors are the most commonly used, indicating their widespread popularity among consumers. This information can help us narrow down our search for popular laptop models that offer a good balance of performance and affordability.

Furthermore, the analysis of laptop prices by brand provides valuable insights. The bar chart displaying the average price of laptops by brand reveals that Razer, Gigabyte, and Jtd are the brands with the highest average prices, while Corsair, Latitude, and Shoxlab offer more budget-friendly options. By considering these brand price ranges, users can make informed decisions based on their budget constraints and desired laptop specifications.

Lastly, the data on RAM popularity and average price per RAM size can assist in optimizing cost savings. The popularity of different RAM sizes shows that 32GB, 16GB, and 64GB are the most commonly used options. However, the average price per RAM size indicates that 16GB RAM offers a good balance between performance and cost, making it a cost-effective choice for most users.

In conclusion, by examining the data, we can optimize laptop components for cost savings by selecting processors like Intel Core i5 and Ryzen 5, exploring affordable prices on eBay, considering popular laptop models with Intel Core i5 and i7 processors, evaluating brand price ranges, and choosing 16GB RAM for a balance between performance and cost. These insights will help users make informed decisions when purchasing laptops, ensuring they get the best value for their money.

5 References and Resources

5.1 References:

What is a Laptop: <https://www.lenovo.com/ph/en/glossary/what-is-laptop/?orgRef=https%253A%252F%252Fwww.google.com%252F>

History of computers: https://en.wikipedia.org/wiki/History_of_laptops

E-commerce Statistics :<https://www.kaggle.com/datasets/keremabdullahoglu/laptop-price-dataset-april-2024>

5.2 Resources:

eBay Dataset: <https://www.kaggle.com/datasets/keremabdullahoglu/laptop-price-dataset-april-2024>

Amazon Dataset:<https://www.kaggle.com/datasets/talhabarkaatahmad/laptop-prices-dataset-october-2023>